Projet sur la Classification des images avec un réseau de neurones – Nebras Triki

1. Contexte et objectifs du projet :

La reconnaissance d'images représente l'un des domaines les plus courants en intelligence artificielle.Dans ce projet de **Deep Learning** , notre but principal est la classification automatique d'images de vêtements à partir du Dataset **FashionMNIST** en dix catégories distinctes à travers la création , l'entrainement et l'évaluation d'un réseau de neurones entièrement connecté (Fully Connected Neural Network (**FNN**)) à l'aide de la bibliothèque **Pytorch**.

2. Données et prétraitement :

Le Dataset **FashionMNIST** contient 60 000 images en niveaux de gris 28*28 , couvrant 10 classes de vêtements (Sneacker , Bag , Ankle boot , T-shirt , Trouser , Pullover , Coat , Dress , Shirt , Sandal) . On a divisé les données en 48 000 images pour l'entrainement et le reste (12 000) pour la validation, en appliquant un split 80/20 via random_split . Ce split garantit une évaluation non biaisée des performances. Pour notre **Pipeline**, chaque image est convertie en tenseur avec ToTensor() , puis chargée par batchs de 64 (batch_size=64) via DataLoader avec mélange aléatoire activé pour l'entrainement , désactivé pour la validation (shuffle=True ou False) .

3. Architecture du modèle :

Le modèle **FashionClassifier** est un réseau qui hérite de nn.Module et repose sur une architecture fully connected séquentielle. Il est composé de trois couches linéaires (Linear) avec l'activation ReLU :

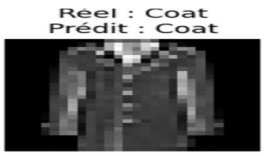
Couche	Description
Flatten : Couche d'entrée : 784 neurones	28*28 ->vecteur de 784 valeurs
Linear : Couche cachée 1 : 128 neurones + ReLU (De 784 à 128)	Première couche d'apprentissage : Extraction de représentations intermédiaires)
Linear : Couche cachée 2 : 64 neurones + ReLU (De 128 à 64)	Compresssion et abstraction
Linear : Couche de sortie :10 neurones (une par classe) (De 64 à 10)	Sortie : un score par classe (logits)

->La fonction de perte standard choisie est CrossEntropyLoss car elle est adaptée à la classification multi-classes et l'optimiseur adaptif Adam avec un taux d'apprentissage de lr=0.001 nous assure une convergence rapide et stable.

4. Entrainement et résultats :

L'entrainement du modèle suit le **Pipeline** classique : remise à zéro des gradients par zero_grad , puis, forward pass, ensuite, calcul de la perte suivi par backpropagation : backward pass et la mise à jour des poids : optimizer.step . Exemple : Le modèle a été entrainé sur 5 epochs, les résultats obtenus sont présentes dans l'images suivantes :

```
Epoch [1/5], Loss: 0.5851
Epoch [2/5], Loss: 0.4067
Epoch [3/5], Loss: 0.3629
Epoch [4/5], Loss: 0.3349
Epoch [5/5], Loss: 0.3132
Accuracy sur les données de validation : 87.42%
```





->La perte diminue régulièrement à chaque epoch, ce qui indique un apprentissage stable et sans divergence. Sur les données de validation, le modèle atteint une précision de 87.42% avec des prédictions correctes vérifiées visuellement sur plusieurs exemples comme présenté en comparant la classe réelle et la classe prédite (ex : Coat , Dress ...) , ce qui démontre une bonne capacité de généralisation du modèle sur des données jamais vues.

5. Conclusion et perspectives :

Ce projet couvre le **Pipeline** complet d'un projet de **Deep Learning** : préparation des données (chargement et split) , conception et entrainement d'un réseau de neurones , évaluation et visualisation des prédictions (performances) . Il est limité aux fondamentaux . Plusieurs améliorations de résultats sont envisageables :

-Remplacer le **FCN** par un **CNN** (un réseau convolutif) , nettement plus adapté aux données spatiales.

-Augmenter le nombre d'epochs ou appliquer du data augmentation.

-Intégrer du dropout pour réduire le surapprentissage, ou appliquer un lr scheduler pour renforcer la généralisation.

-Introduire des métriques d'évaluation avancées (metrics comme **F1-score**, **matrice de confusion**) et tracer les courbes **training et validation loss** avec **Matplotlib** pour détecter et corriger le surapprentissage.